

Inigo Quilez :: articles :: storing floating point in 4k - 2008



[originally written for [in 4k](#)]

As we all know floating point values are bad for 4k intros. If you ever examined the /verbose output of [Crinkler](#), you probably noticed that they are compress to 4 bytes, meaning they don't actually compress. So any attempt to get rid of them is a good thing to try. For example, geometry, camera positions and so on can very well be encoded in 6 to 8 bits. But there is still a few floats that one can not avoid, like projection matrices and other data that your preferred 3D acceleration API only accepts as floats.

For this data I did read somewhere that in The Product they got quite a lot of space saving by truncating the mantissa of the floating point data. Recently Shash/Collapse confirmed me that it also helps in 4k as expected. Apparently Slack/Necrostudios made a script in I don't know what language that takes a .cpp as input, uses regular expressions to detect floating point numbers on it and rewrites the .cpp with new floating point values, close enough to the original, with the last 16 bits of the mantissa reset to zero.

I never made a single Linux script in my life, so I cannot afford now to learn Python, Perl or whatever to do the job. So instead, I just hacked a small program that generates a file with some floating points constants ready to use in my 4k intros. Since most of the time I use floats to adjust formulas (for colors, animations, etc), my constants often live in the (-2,2) range, so I just computed values in that range. When I have a float value out of that range, I always try to make it end in .0f, .5f, .25f, .125f and so on.

Anyway, the thing works quite well, you can check it in the verbose output of Crinkler (half the float data is zeros). Crinkler compresses these constants to something between 1.8 and 2 bytes (half the size, yeah!). I still don't know why it doesn't compress the leading 3E/3F too, giving a 1byte/float, but well, and improvement of 2 bytes per float is not bad anyway :) Here goes the file I have, just in case anyone wants to use it. Note, the last version of Crinkler does automatic bit truncation for floating point constants, but it only works for arrays of data and other special situations where the linker knows the data is floating point. It doesn't work in the general case, so this tricks still apply.

```
//-----//
// File generated automatically, code written by iq/rgba 2007 //
//-----//
#ifdef _FPCONSTANTS_H_
```

```
#define _FPCONSTANTS_H_
#define p0d00 0.0000000000f // 0.00f 0x00000000
#define p0d01 0.0100097656f // 0.01f 0x3c240000
#define p0d02 0.0200195313f // 0.02f 0x3ca40000
#define p0d03 0.0300292969f // 0.03f 0x3cf60000
#define p0d04 0.0400390625f // 0.04f 0x3d240000
#define p0d05 0.0500488281f // 0.05f 0x3d4d0000
#define p0d06 0.0600585938f // 0.06f 0x3d760000
#define p0d07 0.0698242188f // 0.07f 0x3d8f0000
#define p0d08 0.0800781250f // 0.08f 0x3da40000
#define p0d09 0.0898437500f // 0.09f 0x3db80000
#define p0d10 0.1000976563f // 0.10f 0x3dcd0000
#define p0d11 0.1098632813f // 0.11f 0x3de10000
#define p0d12 0.1201171875f // 0.12f 0x3df60000
#define p0d13 0.1298828125f // 0.13f 0x3e050000
#define p0d14 0.1396484375f // 0.14f 0x3e0f0000
#define p0d15 0.1503906250f // 0.15f 0x3e1a0000
#define p0d16 0.1601562500f // 0.16f 0x3e240000
#define p0d17 0.1699218750f // 0.17f 0x3e2e0000
#define p0d18 0.1796875000f // 0.18f 0x3e380000
#define p0d19 0.1904296875f // 0.19f 0x3e430000
#define p0d20 0.2001953125f // 0.20f 0x3e4d0000
#define p0d21 0.2099609375f // 0.21f 0x3e570000
#define p0d22 0.2197265625f // 0.22f 0x3e610000
#define p0d23 0.2304687500f // 0.23f 0x3e6c0000
#define p0d24 0.2402343750f // 0.24f 0x3e760000
#define p0d25 0.2500000000f // 0.25f 0x3e800000
#define p0d26 0.2597656250f // 0.26f 0x3e850000
#define p0d27 0.2695312500f // 0.27f 0x3e8a0000
#define p0d28 0.2792968750f // 0.28f 0x3e8f0000
#define p0d29 0.2890625000f // 0.29f 0x3e940000
#define p0d30 0.3007812500f // 0.30f 0x3e9a0000
#define p0d31 0.3105468750f // 0.31f 0x3e9f0000
#define p0d32 0.3203125000f // 0.32f 0x3ea40000
#define p0d33 0.3300781250f // 0.33f 0x3ea90000
#define p0d34 0.3398437500f // 0.34f 0x3eae0000
#define p0d35 0.3496093750f // 0.35f 0x3eb30000
#define p0d36 0.3593750000f // 0.36f 0x3eb80000
#define p0d37 0.3691406250f // 0.37f 0x3ebd0000
#define p0d38 0.3808593750f // 0.38f 0x3ec30000
#define p0d39 0.3906250000f // 0.39f 0x3ec80000
#define p0d40 0.4003906250f // 0.40f 0x3ecd0000
#define p0d41 0.4101562500f // 0.41f 0x3ed20000
#define p0d42 0.4199218750f // 0.42f 0x3ed70000
#define p0d43 0.4296875000f // 0.43f 0x3edc0000
#define p0d44 0.4394531250f // 0.44f 0x3ee10000
#define p0d45 0.4492187500f // 0.45f 0x3ee60000
#define p0d46 0.4609375000f // 0.46f 0x3eec0000
#define p0d47 0.4707031250f // 0.47f 0x3ef10000
#define p0d48 0.4804687500f // 0.48f 0x3ef60000
```

```
#define p0d49 0.4902343750f // 0.49f 0x3efb0000
#define p0d50 0.5000000000f // 0.50f 0x3f000000
#define p0d51 0.5117187500f // 0.51f 0x3f030000
#define p0d52 0.5195312500f // 0.52f 0x3f050000
#define p0d53 0.5312500000f // 0.53f 0x3f080000
#define p0d54 0.5390625000f // 0.54f 0x3f0a0000
#define p0d55 0.5507812500f // 0.55f 0x3f0d0000
#define p0d56 0.5585937500f // 0.56f 0x3f0f0000
#define p0d57 0.5703125000f // 0.57f 0x3f120000
#define p0d58 0.5781250000f // 0.58f 0x3f140000
#define p0d59 0.5898437500f // 0.59f 0x3f170000
#define p0d60 0.6015625000f // 0.60f 0x3f1a0000
#define p0d61 0.6093750000f // 0.61f 0x3f1c0000
#define p0d62 0.6210937500f // 0.62f 0x3f1f0000
#define p0d63 0.6289062500f // 0.63f 0x3f210000
#define p0d64 0.6406250000f // 0.64f 0x3f240000
#define p0d65 0.6484375000f // 0.65f 0x3f260000
#define p0d66 0.6601562500f // 0.66f 0x3f290000
#define p0d67 0.6718750000f // 0.67f 0x3f2c0000
#define p0d68 0.6796875000f // 0.68f 0x3f2e0000
#define p0d69 0.6914062500f // 0.69f 0x3f310000
#define p0d70 0.6992187500f // 0.70f 0x3f330000
#define p0d71 0.7109375000f // 0.71f 0x3f360000
#define p0d72 0.7187500000f // 0.72f 0x3f380000
#define p0d73 0.7304687500f // 0.73f 0x3f3b0000
#define p0d74 0.7382812500f // 0.74f 0x3f3d0000
#define p0d75 0.7500000000f // 0.75f 0x3f400000
#define p0d76 0.7617187500f // 0.76f 0x3f430000
#define p0d77 0.7695312500f // 0.77f 0x3f450000
#define p0d78 0.7812500000f // 0.78f 0x3f480000
#define p0d79 0.7890625000f // 0.79f 0x3f4a0000
#define p0d80 0.8007812500f // 0.80f 0x3f4d0000
#define p0d81 0.8085937500f // 0.81f 0x3f4f0000
#define p0d82 0.8203125000f // 0.82f 0x3f520000
#define p0d83 0.8281250000f // 0.83f 0x3f540000
#define p0d84 0.8398437500f // 0.84f 0x3f570000
#define p0d85 0.8515625000f // 0.85f 0x3f5a0000
#define p0d86 0.8593750000f // 0.86f 0x3f5c0000
#define p0d87 0.8710937500f // 0.87f 0x3f5f0000
#define p0d88 0.8789062500f // 0.88f 0x3f610000
#define p0d89 0.8906250000f // 0.89f 0x3f640000
#define p0d90 0.8984375000f // 0.90f 0x3f660000
#define p0d91 0.9101562500f // 0.91f 0x3f690000
#define p0d92 0.9218750000f // 0.92f 0x3f6c0000
#define p0d93 0.9296875000f // 0.93f 0x3f6e0000
#define p0d94 0.9414062500f // 0.94f 0x3f710000
#define p0d95 0.9492187500f // 0.95f 0x3f730000
#define p0d96 0.9609375000f // 0.96f 0x3f760000
#define p0d97 0.9687500000f // 0.97f 0x3f780000
#define p0d98 0.9804687500f // 0.98f 0x3f7b0000
```

```
#define p0d99 0.9882812500f // 0.99f 0x3f7d0000
#define p1d00 1.0000000000f // 1.00f 0x3f800000
#define p1d01 1.0078125000f // 1.01f 0x3f810000
#define p1d02 1.0234375000f // 1.02f 0x3f830000
#define p1d03 1.0312500000f // 1.03f 0x3f840000
#define p1d04 1.0390625000f // 1.04f 0x3f850000
#define p1d05 1.0468750000f // 1.05f 0x3f860000
#define p1d06 1.0625000000f // 1.06f 0x3f880000
#define p1d07 1.0703125000f // 1.07f 0x3f890000
#define p1d08 1.0781250000f // 1.08f 0x3f8a0000
#define p1d09 1.0937500000f // 1.09f 0x3f8c0000
#define p1d10 1.1015625000f // 1.10f 0x3f8d0000
#define p1d11 1.1093750000f // 1.11f 0x3f8e0000
#define p1d12 1.1171875000f // 1.12f 0x3f8f0000
#define p1d13 1.1328125000f // 1.13f 0x3f910000
#define p1d14 1.1406250000f // 1.14f 0x3f920000
#define p1d15 1.1484375000f // 1.15f 0x3f930000
#define p1d16 1.1562500000f // 1.16f 0x3f940000
#define p1d17 1.1718750000f // 1.17f 0x3f960000
#define p1d18 1.1796875000f // 1.18f 0x3f970000
#define p1d19 1.1875000000f // 1.19f 0x3f980000
#define p1d20 1.2031250000f // 1.20f 0x3f9a0000
#define p1d21 1.2109375000f // 1.21f 0x3f9b0000
#define p1d22 1.2187500000f // 1.22f 0x3f9c0000
#define p1d23 1.2265625000f // 1.23f 0x3f9d0000
#define p1d24 1.2421875000f // 1.24f 0x3f9f0000
#define p1d25 1.2500000000f // 1.25f 0x3fa00000
#define p1d26 1.2578125000f // 1.26f 0x3fa10000
#define p1d27 1.2734375000f // 1.27f 0x3fa30000
#define p1d28 1.2812500000f // 1.28f 0x3fa40000
#define p1d29 1.2890625000f // 1.29f 0x3fa50000
#define p1d30 1.2968750000f // 1.30f 0x3fa60000
#define p1d31 1.3125000000f // 1.31f 0x3fa80000
#define p1d32 1.3203125000f // 1.32f 0x3fa90000
#define p1d33 1.3281250000f // 1.33f 0x3faa0000
#define p1d34 1.3437500000f // 1.34f 0x3fac0000
#define p1d35 1.3515625000f // 1.35f 0x3fad0000
#define p1d36 1.3593750000f // 1.36f 0x3fae0000
#define p1d37 1.3671875000f // 1.37f 0x3faf0000
#define p1d38 1.3828125000f // 1.38f 0x3fb10000
#define p1d39 1.3906250000f // 1.39f 0x3fb20000
#define p1d40 1.3984375000f // 1.40f 0x3fb30000
#define p1d41 1.4062500000f // 1.41f 0x3fb40000
#define p1d42 1.4218750000f // 1.42f 0x3fb60000
#define p1d43 1.4296875000f // 1.43f 0x3fb70000
#define p1d44 1.4375000000f // 1.44f 0x3fb80000
#define p1d45 1.4531250000f // 1.45f 0x3fba0000
#define p1d46 1.4609375000f // 1.46f 0x3fbb0000
#define p1d47 1.4687500000f // 1.47f 0x3fbc0000
#define p1d48 1.4765625000f // 1.48f 0x3fbd0000
```

```
#define pld49 1.4921875000f // 1.49f 0x3fbf0000
#define pld50 1.5000000000f // 1.50f 0x3fc00000
#define pld51 1.5078125000f // 1.51f 0x3fc10000
#define pld52 1.5234375000f // 1.52f 0x3fc30000
#define pld53 1.5312500000f // 1.53f 0x3fc40000
#define pld54 1.5390625000f // 1.54f 0x3fc50000
#define pld55 1.5468750000f // 1.55f 0x3fc60000
#define pld56 1.5625000000f // 1.56f 0x3fc80000
#define pld57 1.5703125000f // 1.57f 0x3fc90000
#define pld58 1.5781250000f // 1.58f 0x3fca0000
#define pld59 1.5937500000f // 1.59f 0x3fcc0000
#define pld60 1.6015625000f // 1.60f 0x3fcd0000
#define pld61 1.6093750000f // 1.61f 0x3fce0000
#define pld62 1.6171875000f // 1.62f 0x3fcf0000
#define pld63 1.6328125000f // 1.63f 0x3fd10000
#define pld64 1.6406250000f // 1.64f 0x3fd20000
#define pld65 1.6484375000f // 1.65f 0x3fd30000
#define pld66 1.6562500000f // 1.66f 0x3fd40000
#define pld67 1.6718750000f // 1.67f 0x3fd60000
#define pld68 1.6796875000f // 1.68f 0x3fd70000
#define pld69 1.6875000000f // 1.69f 0x3fd80000
#define pld70 1.7031250000f // 1.70f 0x3fda0000
#define pld71 1.7109375000f // 1.71f 0x3fdb0000
#define pld72 1.7187500000f // 1.72f 0x3fdc0000
#define pld73 1.7265625000f // 1.73f 0x3fdd0000
#define pld74 1.7421875000f // 1.74f 0x3fdf0000
#define pld75 1.7500000000f // 1.75f 0x3fe00000
#define pld76 1.7578125000f // 1.76f 0x3fe10000
#define pld77 1.7734375000f // 1.77f 0x3fe30000
#define pld78 1.7812500000f // 1.78f 0x3fe40000
#define pld79 1.7890625000f // 1.79f 0x3fe50000
#define pld80 1.7968750000f // 1.80f 0x3fe60000
#define pld81 1.8125000000f // 1.81f 0x3fe80000
#define pld82 1.8203125000f // 1.82f 0x3fe90000
#define pld83 1.8281250000f // 1.83f 0x3fea0000
#define pld84 1.8437500000f // 1.84f 0x3fec0000
#define pld85 1.8515625000f // 1.85f 0x3fed0000
#define pld86 1.8593750000f // 1.86f 0x3fee0000
#define pld87 1.8671875000f // 1.87f 0x3fef0000
#define pld88 1.8828125000f // 1.88f 0x3ff10000
#define pld89 1.8906250000f // 1.89f 0x3ff20000
#define pld90 1.8984375000f // 1.90f 0x3ff30000
#define pld91 1.9062500000f // 1.91f 0x3ff40000
#define pld92 1.9218750000f // 1.92f 0x3ff60000
#define pld93 1.9296875000f // 1.93f 0x3ff70000
#define pld94 1.9375000000f // 1.94f 0x3ff80000
#define pld95 1.9531250000f // 1.95f 0x3ffa0000
#define pld96 1.9609375000f // 1.96f 0x3ffb0000
#define pld97 1.9687500000f // 1.97f 0x3ffc0000
#define pld98 1.9765625000f // 1.98f 0x3ffd0000
```

```
#define p1d99 1.9921875000f    // 1.99f    0x3fff0000
#endif
```

As you see the solution is not very elegant, because you have to write your formulas like

```
float y = p0d3 + p0d7*cosf(p1d57*time);    // 0.3f + 0.7f*cosf(1.57079f*time);
```

The good thing is that unlike automatic methods (as the mentioned script), here you do control where you allow for a bit of precision lose in favor of compressibility. If anyone finds a more elegant method, that allows one to do the trick while programming and not as an external tool, tell me please!

inigo quilez - learning computer graphics since 1994